

Stitches, not shots: a machine-checked work-tracking discipline for decomposed LLM generation

Tony Gair · 22 September 2026 Preprint. The Dark Factory Ltd, South Shields, United Kingdom.

Abstract

A language model asked to produce a specification-to-proof artifact in one shot fails in ways that are hard to catch: the result looks plausible, there is no record of what was owed against what was delivered, and “done” is finally the model’s own claim. We report a work-tracking discipline built to remove that last property. An owed unit of work is recorded, *before it is attempted*, as a **mūi** — a named stitch carrying a bounded **scope** and a **deterministic check** — and it is admitted as done only when its check passes, run **off the author’s own machine**, so that no author can mark their own home-work. State is never stored as a flag; it is recomputed by running the check, so a claim that quietly stopped being true reverts to *loose* on its own. We describe the primitive, the lay-before-work rule, the off-author admission gate, and why this decomposition beats one-shotting on error-localisation, provenance, and honesty-under-failure. As evidence we report a working factory whose decision-making components are proven SPARK cores, each forged from prose and admitted off-author, and a public release (Crucible v0.2.0, 22 September 2026) gated on such stitches. We give the finish rates plainly — they are a minority of runs, and a clean-container run on a stranger’s machine with a public model finished none of seven — and argue that the discipline, not the finish rate, is the contribution: when the system says *proved*, an independent checker said so, and when it cannot, it says why.

1. Introduction

Ask a capable model to turn a prose specification into a verified software component, and you will frequently get something that compiles, reads well, and is wrong. The failure has three properties that make it dangerous rather than merely disappointing.

- It is **silent**. A contract weaker than the specification, a proof that establishes nothing, a dropped not in a postcondition — each produces an artifact that *looks* finished.
- It is **unattributable**. A one-shot run leaves no durable record of what was owed versus what was delivered, so a later reader cannot tell a gap from a decision.
- It is **unverifiable by anyone but the model**. Completion rests on the generator’s assertion that it did the job.

The instinct is to reach for a better prompt. We argue the fix is not a better prompt but a better *unit of account*. The mechanisms that “agentic” systems already use to organise multi-step work — to-do lists, task queues, plan-and-execute scaffolds, scratchpad plans — record *intent* and carry *no check*. A checked-off item is still an assertion. What

is missing is a unit of owed work that carries, from the moment it is written down, the machine test that will decide whether it is done, and a rule that the test is run by someone other than the party claiming completion.

We call that unit a **mũi**. The word is Vietnamese for a sewing or embroidery stitch (*mũi khâu*, an embroidery stitch; *mũi kim*, the needle’s point), chosen deliberately over “task” — which in this setting collides with a reserved word of the target language — and chosen to be *spoken*: one can say “six anchored, eleven loose, one frayed” across a room.¹ The states that a stitch moves through keep the English vocabulary of needlework, where it carries the argument: a stitch is **loose** until something holds it, **anchored** once evidence secures it (as anchoring in needlework secures a thread so the work cannot pull out), and it **frays** when a check that once held silently stops holding.

Contributions.

1. The **mũi primitive**: identifier, scope, deterministic check, and a two-state lifecycle whose state is *recomputed* rather than stored (§3).
2. The **lay-before-work discipline**: every owed item in a plan is written as a *mũi*, not as prose, so the plan is a checkable board rather than a narrative (§3–4).
3. The **off-author admission gate**: the check runs off the author’s machine and the author can never self-admit; a signed receipt from an independent checker is what closes a stitch (§4).
4. An **experience report** (§6): a real factory of proven cores built under this discipline, including a public release measured on a stranger’s machine, with the finish rates given unflatteringly.

We answer a narrow question — *does recording owed work as checked stitches, admitted off-author, beat one-shotting a whole artifact?* — with a qualified yes, and we are precise about which part of the chain the discipline does and does not secure.

2. Background and related work

Proof-carrying code. The idea that a producer should spend the effort and a consumer should check a cheap certificate originates with Necula and Lee [1, 2]. The discipline we report is a management-layer cousin: where proof-carrying code attaches a certificate to a *shipped artifact*, we attach a check to a *unit of owed work* and admit the work only when an independent party runs that check. The two compose — a companion report [7] attaches independently-checkable certificates to the SMT step of the same factory’s verification — but the contribution here is the tracking discipline, not the certificate format.

Agentic task tracking. Plan-and-execute and reflective agent architectures — reason-then-act loops, plan-and-solve prompting, self-reflection over a scratchpad, self-consistency over sampled traces [3, 4, 5, 6] — organise multi-step generation

¹The mixed vocabulary — a Vietnamese unit noun with English needlework states — is a deliberate naming choice by the author, honouring the craft the metaphor is borrowed from rather than importing an English word into it. Glosses here are the author’s and have not been checked by a native speaker; that check is owed before the terminology is leaned on in translated material.

well, but their record of progress is a list of *intents*. None attaches to an owed item the deterministic test that will decide whether it is done, and none separates the party that claims completion from the party that verifies it. Marking an item complete remains an assertion by the generator, which is exactly the property that makes one-shot failure silent.

Test-driven development writes the test before the code, and is the closest everyday analogue to lay-before-work. The difference is organisational: TDD’s tests are run by the same developer who writes the code, on the same machine, and passing is self-reported. The *mũi* discipline adds the off-author admission rule — the check is the unit of trust, and it is run by someone who is not the author.

Structured, probabilistic decisions for agents. A contemporaneous line makes agent decisions structured but keeps them *probabilistic*. Jev, a “System-One” classifier from TypeSafe AI [9], is given a state and typed questions and returns typed answers *with probability scores*, at a claimed “up to 200× faster inference and 400× lower cost” than a comparable model on classification tasks; its stated uses are model routing and safety guardrails that evaluate a tool call before it runs. It is the instructive opposite of a *mũi*, and the contrast sharpens the thesis. A Jev answer is a *fast heuristic verdict with a confidence* — System-1; a *mũi*’s admission is a *re-runnable check* — System-2. The difference matters most exactly where it is load-bearing. A probabilistic guardrail decides from a score, and a score can be moved by the very input it is judging; a control that can be starved or steered by the thing it controls is not, in the limit, a control. A *mũi*’s check re-runs on every reading and cannot be talked out of its verdict — that is the whole point of recomputing state rather than storing it (§3). The two compose cleanly, and we do not claim otherwise: a System-1 classifier is the right tool for a cheap first-pass router, and a proven core is the right authority for a decision that must hold. What the thesis insists on is only the boundary — a load-bearing decision, one that another component or a person will rely on, must not finally rest on a probability, however cheap and fast that probability is to obtain. (What “must hold” can be claimed for such a proof, and what it cannot, is treated in §7.)

SPARK and gnatprove as the oracle. SPARK is a verifiable subset of Ada; gnatprove discharges verification conditions with SMT back ends and returns a deterministic verdict. It gives us a check that is a genuine oracle rather than a heuristic: for a core forged from prose, “the prover closed every obligation” is a fact a second machine can re-derive. The separation the discipline enforces — *prove, do not merely generate* — depends on having such an oracle for the domain.

AI-produced verified software. Contemporaneous work reports verified security software produced by AI agents at large scale [8]. That work addresses production of the artifacts; the question here is orthogonal and upstream — how owed work is recorded and admitted while such artifacts are being produced, so that “done” is never the generator’s word.

3. The *mũi* primitive

A *mũi* is four things:

field	what it is
id	a stable, speakable name for the owed claim (e.g. <code>door-is-sole</code>)
scope	the set of paths or artifacts the work is permitted to touch
check	a shell command that exits 0 exactly when the claim holds
state	derived, not stored: loose or anchored (with frayed and dropped for the failure and chain cases below)

Two properties are load-bearing.

The claim is written as what will be *true*, not as the work to do. “The correction is committed” — never “commit the correction.” Work can be performed while the claim remains false (a commit that does not actually apply the fix), so the claim, not the activity, is what the check tests. If you cannot write a check, you do not have a stitch; you have an intention, and the tool records it as exactly that — honestly, every time it is read — rather than letting it masquerade as tracked work.

State is recomputed by running the check, never read from a flag. There is deliberately no “mark done.” Asking the board for status runs every check and reports what is still loose. This is the single property that defeats silent staleness: a claim that was anchored last week and whose check fails today comes back *loose* on its own, with no one having touched it. A prose owed-list is a stored flag — true when written, never re-read; a board of *mũi* marks nothing as its own homework.

Lay before work. The standing rule is that every owed item in a plan or a session note is laid as a *mũi* at the moment one would otherwise write “still owed.” The plan thereby becomes a board rather than a story. Reading the board answers the only question worth asking of a plan — not “what is the status” but “what is outstanding” — and the answer is computed, not recalled.

Chains. A *mũi* may be worked *through* another, chain-stitch fashion: it can be anchored only while everything upstream of it is anchored. A stitch whose own check passes but which sits on a failed upstream stitch is **dropped** — the signal that the repair belongs upstream, not here. Cycles are refused when a stitch is laid. Chains let a release gate — “the artifact is fit to ship” — sit visibly on the specific claims it depends on, so that an upstream regression pulls the whole chain loose rather than leaving a stale green light at the top.

Scope containment. The work admitted under a stitch must touch only paths inside that stitch’s declared scope. A stitch laid to change an emitter cannot carry a change to the front door. This is what lets the board make a second promise beyond “this claim holds”: *the neighbours were provably not touched* while it was made good.

4. The discipline: lifecycle and off-author admission

The lifecycle is deliberately short: **lay** → **work within scope** → **run the check** → **anchor only if it passes**. A failing check does not fail loudly and stop; it simply leaves the stitch loose and honest. There is no path from “I did the work” to “it is done” that does not pass through the check.

The property that distinguishes this from self-tracked TDD is **off-author admission**.

- **The check is run off the author’s machine.** For a forged core, the machine that re-proves it is separate from the one that produced it, and it signs a receipt there. The author’s say-so is not admissible evidence; the receipt is.
- **The author can never self-admit.** The admission gate is decided by a proven component, not by the party doing the work, and the authority to declare a stitch ratified is held outside the author’s toolset entirely. The design question this forces is explicit and was treated as such: *without an external ratifier, a gate that admits “work carried by a stitch” is bypassed simply by laying the stitch* — self-authorisation. The gate is therefore built so that laying a stitch grants no licence; a separate, human-held act ratifies it, and the mutation must additionally lie inside the stitch’s scope.
- **The mutation carries the stitch’s id.** Each admitted change names the stitch it discharges, so the record shows which claim carried which change, and the admission log reads back as a ledger of claims made good rather than of files touched.

We are precise about the strength of this gate. In the current system the re-proving machine is separate from the forging one and signs its own receipt, but the forging party still holds administrative rights on the prover. Until that is removed, the chain of custody is **measured, not attested** — and “measured” is the honest word for it. A demonstrated failure mode sharpens the point: a receipt naming an independent prover can be minted using only environment variables and no privileged access, for a component never actually proved by anyone. The receipt format is therefore producer-asserted unless the prover’s independence is itself enforced; the discipline states this as a limitation rather than papering over it.

5. Why it beats one-shotting

The argument is not that stitches make the model better. It is that they change what has to be trusted.

- **Error localisation.** A one-shot artifact that fails yields a single failed whole. A chain of stitches that fails yields a *named* failed stitch, with its neighbours provably untouched, so the defect is localised to a scope rather than smeared across an artifact.
- **The check is the unit of trust, not the model.** A stitch that “looks done” but whose check fails stays loose. No amount of fluent, plausible output moves it. This directly answers the silent-failure property: plausibility is not admissible; the check is.

- **Monotonic honesty.** Because state is recomputed and there is no mark-done, the record can only move toward truth. A stitch cannot be discharged by assertion, and one that stops holding reverts on its own at the next reading.
- **Provenance.** Every admitted change names its stitch and carries a signed off-author receipt, so “what was owed versus delivered” is a query against the board, not a reconstruction from memory.
- **Bounded blast radius.** Scope containment means a bad stitch cannot silently alter a proven neighbour — the property that makes it safe to keep building on a large body of already-proven work.

property	one-shot generation	mūi discipline
localisation of failure what is trusted	whole artifact the model’s output	named stitch, scoped the check + the off-author prover
provenance of “done”	the model’s claim	signed receipt naming the checker
honesty under failure	plausible-looking pass	stays loose; frays revert on read
reproducibility	re-run the whole generation	re-run the check on any machine

The one property one-shotting has and the discipline sacrifices is *speed to a plausible answer*. That is precisely the property that makes one-shot failure dangerous.

6. Experience report

The factory. The system under discussion builds Ada/SPARK components from prose specifications and refuses to deliver anything it could not prove. It is itself built the way it builds: the components that make its decisions are proven SPARK packages, and each was produced from prose by the factory rather than hand-written. Every one of them entered service through the discipline above — laid as a stitch, forged within scope, re-proved off-author, admitted only on a signed receipt.

Case study — Crucible v0.2.0, public 22 September 2026. The release² is gated on a chain of stitches — that the front door is the sole entry point, that the shipped artifact depends on no absent component, that the exchange path leaks nothing of the author’s — and its deciding parts are proven, stitch-discharged cores. We give the finish figures plainly, because the discipline’s value does not rest on them being flattering.

what was measured	result
Decompose stage alone (two sheets)	50 of 50 after a prompt fix; 32 of 50 before it

²github.com/the-dark-factory/Crucible_AGPL, tag v0.2.0.

what was measured	result
Whole pipeline, prose → proved unit (first A/B, stopped early at 19 of 40 runs)	3 of 10, and 3 of 9
Whole pipeline on one counting specification	2 of 8 — and one of the two was weaker than its specification
Whole pipeline, later 30-run A/B of a candidate improvement	33% versus 43%; inconclusive, not adopted
Clean container on a stranger’s machine, public model, nothing of ours copied	0 of 7

The whole pipeline finishes in a *minority of runs*; the main stopper is contract emission, where the model exhausts its round budget before writing a contract the prover accepts. On a stranger’s machine, in a clean container, with a public model and none of our tuning, it finished none of seven attempts. We report these rather than omit them because they are the honest frame for the claim, which is narrower and, we think, more useful: **when the system says proved, an independent prover said so; when it cannot, it names the stage it stopped at and keeps the case rather than discarding it.**

The finding that most vindicates the discipline is a failure it caught in itself. Twice in two days the model emitted a contract *weaker* than the specification asked for. One passed every gate in place, including a check for contracts that assert nothing; the other dropped a single not, which would have left a case pending forever, and was caught only by luck through an unrelated syntax error. This is the discipline’s own limitation made concrete (§7): a stitch is only as honest as its check, and a check that tests “the body meets the contract” does not test “the contract is the one you meant.” The discipline surfaced the gap precisely because it forces the check to be named and examined; a one-shot run would have shipped the plausible artifact.

7. Limitations and when a mūi is the wrong tool

We state these plainly; the discipline is easy to over-read.

1. **A stitch is only as honest as its check.** The whole method reduces trust to the quality of the check. A check that passes a proof-of-nothing gives false comfort. The weaker-than-specification contracts in §6 are this failure in the wild: “proved” meant the body met the contract the system *wrote*, not the contract the author *meant*. A check that compares a contract’s strength to a ground truth is designed and not yet built; until it is, a human must read the emitted contract.
2. **The off-author gate is measured, not attested.** As in §4, the prover’s independence is not yet enforced against a producer with administrative rights, and a valid-looking receipt can be minted without a real proof. This is the load-bearing open problem for the trust claim.
3. **Some work resists decomposition, and some checks are undecidable or expensive.** Where an owed claim has no cheap deterministic test — anything turning on exact floating-point behaviour, for instance, is refused at intake here rather

than failed at proof — a stitch is the wrong tool, and the honest move is to say so rather than to lay a stitch whose check cannot answer (which the board reports as *frayed*).

4. **The cost of a good check is real.** Writing the check is often as much work as the task, and the discipline front-loads that cost. It pays back in honesty-under-failure and provenance, not in speed to a first answer.
5. **The evidence is small-N and partly self-measured.** Finish rates are from a handful of sheets; the stranger-machine figure is a single clean-container run; and the chain of custody is measured, not attested. We claim the discipline, not generality of the numbers.
6. **A proof here is not qualification evidence.** The proofs are real and re-runnable, but the toolchain that produced them is not a *qualified* toolchain in the sense that a certified safety-critical programme requires (DO-178C, EN 50128, ISO 26262 each rest on a qualified tool with its own artefacts, traceability and a vendor carrying liability). A stitch’s receipt is therefore something a reader can re-derive; it is *not* something that can be presented as qualification evidence. Such a core may serve as an *input* to a certified programme, but not as a substitute for its qualification. When §5 says a decision “must hold,” this is the ceiling on what that claim reaches.

8. Conclusion

A stitch with a check is a better unit of account than a task with a claim. Recording owed work as a named, scoped, deterministically-checked stitch — laid before the work, its state recomputed on every reading, and admitted only off the author’s own machine — converts an unverifiable one-shot generation problem into a sequence of independently checkable ones. It does not make the model finish more often; on our numbers it finishes a minority of the time. What it makes reliable is the *meaning of “done”*: never the generator’s assertion, always a check that a second machine can re-run. That is what let a factory of proven cores be built, and released, by a language model without the release resting on the model’s word for any of it.

Artifact and reproducibility

The public release referenced in §6 ships sources, a proof summary, and a script that re-checks each core on the reader’s machine. The finish figures in §6 are the author’s own measurements at pinned versions; the clean-container figure is reproducible by any reader from the public release with a public model.

References

- [1] G. C. Necula. *Proof-Carrying Code*. POPL ’97, pp. 106–119. DOI: 10.1145/263699.263712.
- [2] G. C. Necula and P. Lee. *Safe Kernel Extensions Without Run-Time Checking*. OSDI ’96.

- [3] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan and Y. Cao. *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR 2023.
- [4] L. Wang, W. Xu, Y. Lan, Z. Hu, Y. Lan, R. K.-W. Lee and E.-P. Lim. *Plan-and-Solve Prompting*. ACL 2023.
- [5] N. Shinn, F. Cassano, B. Labash, A. Gopinath, K. Narasimhan and S. Yao. *Reflexion: Language Agents with Verbal Reinforcement Learning*. NeurIPS 2023.
- [6] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery and D. Zhou. *Self-Consistency Improves Chain of Thought Reasoning in Language Models*. ICLR 2023.
- [7] T. Gair. *Proof-Carrying Verification Conditions for SPARK: An Experience Report*. Preprint, The Dark Factory Ltd, August 2026. DOI: 10.5281/zenodo.22902004.
- [8] J. Philipp. *The Prover Is the Judge: Verified Security Software from AI Coding Agents in Ada/SPARK*. arXiv:2607.14340, July 2026.
- [9] TypeSafe AI. *Jev: a System-One model for structured decisions / Building a harness with Jev*. langchain.com/blog/building-a-harness-with-jev, 2026. Product announcement; the speed and cost figures (“up to 200× faster inference and 400× lower cost”) are the vendor’s own claims on classification tasks, cited as such.

Bibliographic note. Entries [1], [2], [7] and [9] were verified against primary sources (entry [9] against the vendor’s announcement; its performance figures are the vendor’s claims, not independently measured here). Entries [3]–[6] and [8] should be confirmed against the publishers’ records (venue, author lists, page numbers) before any submission beyond a preprint deposit.

Tooling and AI-use statement

This work was carried out, and this report drafted, in an AI-assisted working session using Anthropic’s Claude (Claude Code). The author is neurodivergent and works remotely; AI assistance is used as an accessibility accommodation, in the way another researcher might work with a scribe.

Per prevailing venue policy an AI system is not an author of this work and is not credited as one. The author takes full responsibility for its contents. The measurements in §6 were produced by named tools and are reproducible from the public release independently of how the drafting session was conducted; the judgements in §4 and §7 — what the discipline does *not* secure, and where trust actually remains — are stated as narrowly as the author could manage, precisely because AI assistance makes overclaiming fluent and easy, and they are the author’s.